Università di Pisa

Master of Science in Computer Engineering

LARGE-SCALE AND MULTI-STRUCTURED DATABASES

BookSphere
Books Review Social Network

Authors:
Danilo Parisi
Matteo Iori
Matteo Billi Ciani

A.Y. 2025–2026

Contents

Chapter 1

Introduction

Welcome to **BookSphere**, the ultimate social platform for book lovers designed to help you organize your reading life and connect with a global community of readers like you.

1.1 Project Overview

Imagine a place where your personal library is always at your fingertips, curated exactly how you live it—sorting your journey from the books you dream of reading, to the ones currently on your nightstand, to the stories that have already shaped you.

But a library should not be a static place. BookSphere allows you to build a circle of literary companions by following friends and fellow readers. Your feed transforms into a living book club, where you can track what your network is devouring in real-time, share reactions, and draw inspiration from the shelves of those you know best.

1.2 Motivation and Key Features

Beyond your personal circle, the motivation behind BookSphere is to connect users to a pulse of like-minded individuals, filtering out the noise of generic popularity. The platform introduces unique features designed to reward quality engagement:

- **Real Influencers:** We surface voices trusted not for their volume, but for the quality of their insight identifies, consistency and knowledge of a genre. Expert reviewers identified by the quality of their engagement rather than just follower count.
- **Trending Probability:** Through advanced data analysis, we help you discover the next literary phenomenon before it hits the mainstream charts.
- **Internationality Index:** We visualize how a story travels across borders, showing you that the book in your hands is being turned by another reader in Tokyo, New York, or Rome at this very moment.

- **Yearly Wrapped:** Ultimately, BookSphere is about your story. As you interact and rate, the platform aggregates your history to generate a personalized statistical summary at the end of the year—visualizing your top authors, favorite genres, and reading milestones.

Chapter 2

System Design

This chapter outlines the architectural blueprint of **BookSphere**. We begin by describing the high-level system design and technology stack, followed by a detailed analysis of functional and non-functional requirements, and concluding with the visual interface design.

2.1 System Architecture

The BookSphere platform is built upon a robust **3-Tier Architecture**, designed to ensure separation of concerns, scalability, and maintainability.

2.1.1 Architectural Layers

- **Presentation Layer (Frontend):** The client-side application is designed to be responsive and intuitive. It communicates with the backend via RESTful APIs, handling user interactions and data visualization (e.g., rendering the social graph or library shelves).
- **Logic Layer (Backend):** The core of the system is developed using **Java** with the **Spring Boot** framework. This layer handles business logic, security (Authentication/Authorization), and acts as an orchestrator between the two different databases. We utilize **Lombok** to reduce boilerplate code and ensure cleaner class definitions.
- **Data Layer (Polyglot Persistence):** The system adopts a hybrid storage strategy:
  - **MongoDB:** Acts as the primary document store for Books, User Profiles, and detailed Reviews.
  - **Neo4j:** Serves as the high-performance graph engine to manage social connections (*FOLLOWS*) and interaction patterns (*READS*, *LIKED*) for the recommendation algorithms.

2.2 Requirements Analysis

2.2.1 Main Actors

The system serves three main categories of users, each with distinct privileges and interaction capabilities:

- **Administrator:** The super-user responsible for platform management. Their tasks range from maintaining the consistency of the catalog (adding/updating books and authors) to community moderation (deleting content and banning users).
- **Registered User:** The primary target audience of the platform. These users can fully interact with the system by creating social connections, writing reviews, curating their library, and generating personal analytics.
- **Unregistered User (Guest):** Users who access the platform in *"read-only"* mode. They can explore the catalog, view public rankings and reviews, but cannot perform active interactions (rating, following, posting).

2.2.2 Functional Requirements

The functional requirements define the specific behaviors and functions of the system, categorized by the actor scope.

General & System Requirements

These requirements apply to the core logic of the platform and are available to all users (including Guests) or performed automatically by the system background processes.

Search & Discovery

1. The System must permit users to search for books and authors using different filters (genre, year, name).
2. The System must allow users to search for other profiles and view their reviews.
3. The System must enable any user to read reviews associated with a book.

System Analytics & Computation

4. The System must compute, for every newly added book, a probability score indicating the likelihood of the book going viral.
5. The System must compute a metric representing an author's versatility based on the variety of genres covered in their works.

6. The System must compute a "How Far a Book Travels" index, measuring the geospatial distribution of a book's readers across the globe.
7. The System must generate various rankings, such as top books or authors, filtered by year or genre.
8. The System must calculate and display the average rating for authors and books for every year.
9. The System must identify trending books based on the volume of reviews and ratings in the current month.
10. The System must automatically identify "Influencers" within the application based on engagement quality metrics.

Access

11. The System must enable an Unregistered User to create a new account (Sign-up).

Registered User Requirements

These features are exclusive to authenticated users.

Personalization & Insights

1. The System must provide book recommendations based on the user's interaction history (favorite genres, authors) and global trends.
2. The System must generate a "Wrapped" style summary of the user's reading year (statistics, top genres).
3. The System must enable users to view a complete list of publications for a specific author.
4. The System must allow users to retrieve all reviews for a specific book on demand.

Content Creation & Library Management

5. The System must allow a user to review books, providing a numerical evaluation (score) and an optional written comment.
6. The System must allow each user to assign a status to a book, selecting from: *To-Read*, *Reading*, or *Read*.
7. The System must enable users to create and manage custom lists of books.

Social Interaction

8. The System must enable users to follow other profiles and view their latest activities in a dedicated feed.
9. The System must enable users to "like" or "unlike" specific books.
10. The System must enable users to "like" or "unlike" reviews written by others.
11. The System must allow users to view their list of friends/followed accounts.

Account Management

12. The System must permit users to Log In and Log Out securely.
13. The System must allow users to delete their account.

Administrator Requirements

Privileged operations for content and user management.

Catalog Management

1. The System must enable an Admin to add new books, authors, or genres to the database.
2. The System must enable an Admin to update existing information related to books, authors, or genres.
3. The System must enable an Admin to remove a specific book, author, or genre.

Moderation & Oversight

4. The System must allow an Admin to view the profile and data of any Registered User.
5. The System must enable an Admin to view any review posted on the platform.
6. The System must enable an Admin to delete reviews deemed inappropriate.
7. The System must enable an Admin to ban a Registered User[1].

[1]Banning prohibits access but preserves specific user metadata to prevent re-registration with the same credentials.

2.2.3 Non-Functional Requirements

These requirements define the quality attributes of the system.

1. The System must follow RESTful design principles for API communication.
2. The System must implement backup and redundancy strategies to avoid permanent data loss.
3. The System must encrypt sensitive user data, specifically passwords, using industry-standard hashing algorithms.
4. The System must be highly available and fault-tolerant to ensure continuous service.
5. The System must enforce an Eventual Consistency model between the Document Database (MongoDB) and the Graph Database (Neo4j) to optimize performance over strict synchronization.

2.3 Expected Workload Distribution

A critical aspect considered when designing the system and database architecture is the expected workload profile. Given the social nature of the application, BookSphere is designed to handle a **Read-Heavy** workload, typical of large-scale social platforms.

2.3.1 Workload Profile (OLTP vs. OLAP)

The system operates primarily as an **OLTP (Online Transaction Processing)** environment, characterized by a high volume of short, atomic transactions (e.g., posting a review, following a user). However, features such as *Yearly Wrapped* and *Trending Probability* introduce periodic **OLAP (Online Analytical Processing)** workloads, requiring the aggregation of vast datasets.

The operational split is estimated as follows:

- **Read Operations (∼80-85%):** The majority of traffic consists of content consumption: browsing book details, traversing the social graph (viewing friends' activities), and authentication.
  - *Architectural Implication:* This imbalance justifies the use of **Replica Sets** in MongoDB. By setting the read preference to `secondaryPreferred`, we offload the heavy read traffic to secondary nodes, keeping the Primary free for write operations.
- **Write Operations (∼15-20%):** Writes are further categorized based on frequency and complexity:
  1. **High-Frequency Writes (∼10-15%):** Lightweight interactions such as 'Likes' and 'Follows'. These require low latency and can tolerate *Eventual Consistency*.

2. **Low-Frequency Writes (<5%):** Content creation tasks such as writing detailed Reviews, rating books, and Administrative operations (CRUD on the catalog). These may require stricter validation.

2.4 UML Class Diagram

To provide a comprehensive view of the system's static structure and the relationships between entities, we present the UML Class Diagram in Figure 2.1.

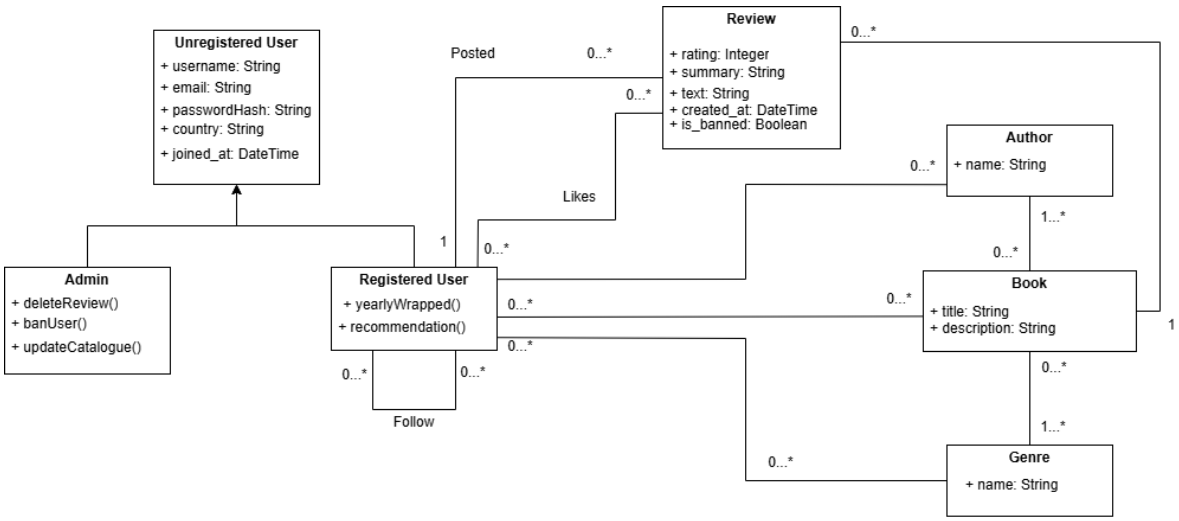


Figure 2.1: UML Class Diagram of BookSphere.

2.5 User Interface Concept and Mock-ups

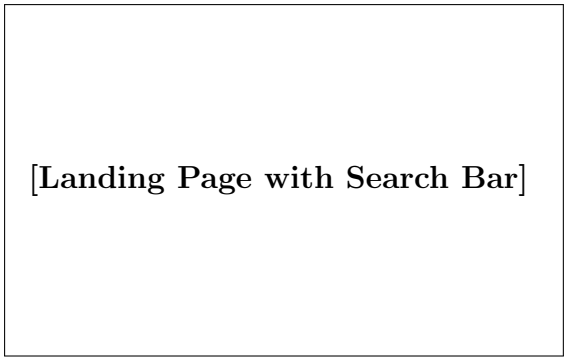
This section presents the mock-ups developed to provide a conceptual visualization of the system's user interface. These designs serve as a bridge between the functional requirements analyzed in the previous section and the final API implementation.

> ***Note:*** *These mock-ups are intended for informational purposes only and were not implemented in the final system due to the project's focus on API-based architecture. However, they demonstrate how the defined requirements would translate into a frontend user experience.*

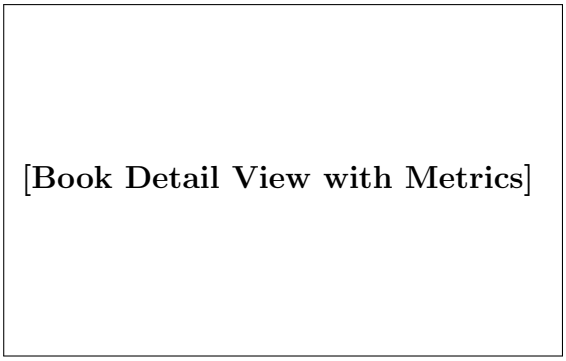
The following figures illustrate the interface design for the three main system actors: **Unregistered Users**, **Registered Users**, and **Administrators**.

2.5.1 Unregistered User Experience

The Guest interface focuses on **Search & Discovery** and **System Analytics**, allowing users to explore the catalog without barriers before converting via the **Access** requirements.



(a) Landing Page & Discovery
Satisfies General Req 1 (Search) & Req 11 (Sign-up)

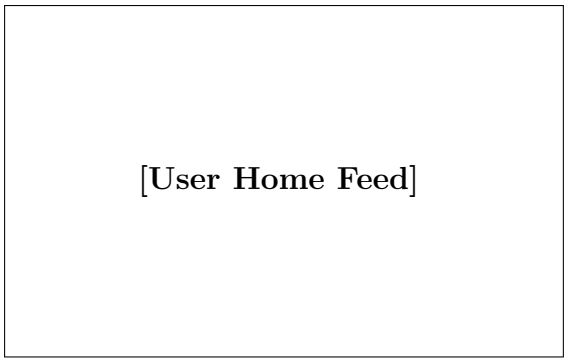


(b) Public Analytics View
Satisfies General Req 4 (Viral Score) & Req 9 (Yearly Ratings)

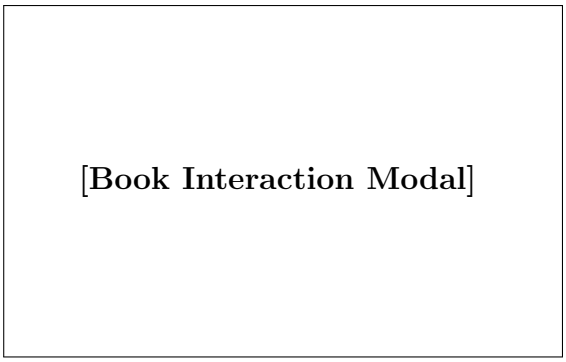
Figure 2.2: Guest interface focusing on discovery and public data visualization.

2.5.2 Registered User Experience

The Authenticated User interface enables **Personalization**, **Content Creation**, and **Social Interaction**. The designs below highlight the transition from passive reading to active engagement.



(a) Social Feed & Recommendations
Satisfies Reg-Req 1 (Recs) & Req 8 (Follow Feed)



(b) Review & Status Management
Satisfies Reg-Req 5 (Review/Score) & Req 6 (Status: To-Read/Read)

Figure 2.3: Core interaction screens for Registered Users.

[**User Year-in-Review "Wrapped" Design**]

Figure 2.4: Personalized Analytics Dashboard.
Satisfies Reg-Req 2 ("Wrapped" Summary)

2.5.3 Administrator Experience

The Admin interface prioritizes **Catalog Management** and **Moderation**. These views are data-dense to facilitate efficient system oversight.

[**Catalog Management Table**]

(a) Database Management
Satisfies Admin Req 1-3 (Add/Update/Remove)

[**User Moderation Panel**]

(b) Moderation & Banning
Satisfies Admin Req 7 (Ban User) & Req 6 (Delete Review)

Figure 2.5: Administrative panels for maintaining system integrity.

Chapter 3

Data Modeling

This chapter details the data persistence layer of **BookSphere**. Given the polyglot nature of the architecture, the data modeling strategy is split between a **Document-Oriented** approach (*MongoDB*) for content management and analytics, and a **Graph-Oriented** approach (*Neo4j*) for social interactions and recommendation algorithms.

3.1 Data Source and Volume

To ensure the system was tested against realistic data volumes and complex interaction patterns, we implemented a hybrid data generation strategy using custom automation scripts. The core catalog and foundational rating data are sourced from the **Amazon Books Reviews Dataset** (providing real-world book metadata and user ratings).

This real data is a user base of approximately 10,000 active accounts. The generation process automatically populates the polyglot persistence layer:

- **Amazon Books Reviews Dataset (∼5GB):** Provides the core catalog of books and real-world reviews spanning from 1996 to 2014.
- **Book-Crossing Dataset:** Used as a supplementary source for user ratings to increase data *variety* and *volume*.

3.2 Document Store: MongoDB

MongoDB acts as the primary "Source of Truth" for the system. It stores the complete information about books, users, and reviews. The schema design prioritizes **Read Performance** through specific NoSQL patterns.

3.2.1 Collections Overview

The database `booksphere_db` contains the following main collections:

MongoDB (Document Store)	Neo4j (Graph Store - CSVs)
• 37.1MB for `reviews` • 27.7MB for `books` • 12.4MB for `users` • 3.5MB for `authors`	• 4.32MB for `Nodes (User, Book, Review...)` • 24.9MB for `Relationships (LIKES, FOLLOWS...)`

Table 3.1: Database Volumes by Collection.

1. **books:** The central catalog entity.
2. **users:** Stores authentication data and the user's library.
3. **reviews:** Stores the full text of reviews. Linked to books and users via references.
4. **authors:** Stores aggregated statistics about authors (e.g., total ratings).

3.2.2 Schema Design (JSON)

Books Collection

We implemented the **Bucket Pattern** (`stats_per_year`) to make historical ranking queries instant, avoiding the need to scan millions of review documents. The **Snapshot Pattern** (`recent_reviews_snapshot`) allows the frontend to render the book details page with the latest reviews in a single query.

```
{
  "_id": "698db76b4a1f5c8fa49331a8",
  "title": "The Fellowship of the Ring",
  "publication_year": 2000,
  "description": "Begin your journey into Middle-earth... The
     inspiration for the upcoming...",
  "availability": "ACTIVE",
  "source": "amazon_master",

  "author": {
    "id": "698db76b4a1f5c8fa4933176",
    "name": "J. R. R. Tolkien"
  },

  "genres": [
    "Fiction"
  ],

```

```
  "external_ids": {
    "isbns": [
      "978-0547928210",
      "..."
    ]
  },

  "recent_reviews_snapshot": [
    {
      "_id": "698db7874a1f5c8fa493ddcf",
      "user_id": "698db7854a1f5c8fa493cad6",
      "username": "User_278550",
      "rating": 98,
      "summary": "",
      "date": "2025-12-27T00:00:00.000+00:00"
    }
    // ... other four recent reviews
  ],

  "popular_reviews_snapshot": [
    {
      "_id": "698db7ba4a1f5c8fa4940742",
      "user_id": "698db7854a1f5c8fa493bd2c",
      "username": "User_AASZHOEVN5LZH",
      "rating": 82,
      "num_of_like": 29,
      "summary": "The legacy of \"The Hobbit\"",
      "date": "2014-02-26T01:00:00.000+00:00"
    }
    // ... other two popular reviews
  ],

  "stats_per_year": [
   {
      "year": 2011,
      "ratings_count": 5,
      "sum_rating": 90
    }
    // ... between years
    {
      "year": 2025,
      "ratings_count": 1,
      "sum_rating": 98
    }
  ],

  "review_ids": [
    "698db7874a1f5c8fa493ddcf",
    "698db7874a1f5c8fa493e262",
    "698db7874a1f5c8fa493e3b8",
    "698db7ba4a1f5c8fa494072c",
```

```
    "..."
  ],

  "month_score": {
    "rating_count": 1,
    "sum_rating": 98,
    "Current_Month": "2025-12"
  }
}
```

Listing 3.1: Example Book Document

Users Collection

The user document utilizes the **Subset Pattern** for the `bookshelf` to keep the document size manageable. Crucially, we maintain a `reviews_year` array (embedded) which is optimized specifically for the **Yearly Wrapped** aggregation, allowing the system to generate the report without joining the `reviews` collection.

```
{
  "_id": "ObjectId('65d1...')",
  "username": "User_12345",
  "email": "user@bx.com",
  "password_hashed": "...",
  "status": "active",

  "bookshelf": [
    {
      "book_id": "ObjectId('65b3f...')",
      "status": "read",
      "added_at": "ISODate('2023-06-01...')",
      "title": "The Hobbit",
      "author": { "name": "J.R.R. Tolkien" },
      "genres": ["Fantasy"]
    }
  ],

  "reviews_year": [
    {
      "id": "ObjectId('99a1...')",
      "rating": 85,
      "book": "Dune"
    }
  ]
}
```

Listing 3.2: User Document Structure

Reviews Collection

The reviews collection stores the full textual content of user evaluations. It serves as the primary source for the "read more" functionality and provides the data for both the `recent_reviews_snapshot` in the books collection and the `reviews_year` array in the user documents.

```
{
  "_id": "698db7874a1f5c8fa493ddcf",
  "user_id": "698db7854a1f5c8fa493c8a5",
  "username": "Rusneus003",
  "rating": 83,
  "source": "amazon",
  "text": "This is, all-around, a pretty good lawn book. However,
      there are a few cases where the advice is a little outdated
     ...",
  "summary": "Pretty good, although a few flaws",
  "created_at": "2022-04-26T02:00:00.000Z",
  "likes_count": 6,
  "is_banned": false,
  "book_snapshot": {
    "book_id": "698db76e4a1f5c8fa493a4ec",
    "title": "The Impatient Gardener's Lawn Book"
  },
  "author_snapshot": {
    "id": "698db76e4a1f5c8fa493a4ed",
    "name": "Jerry Baker"
  }
}
```

Listing 3.3: Example Review Document

Authors Collection

The authors collection is designed to facilitate quick retrieval of author profiles and their complete bibliography. It utilizes the **Extended Reference Pattern** to list published books and the **Computed Pattern** to maintain real-time aggregate statistics.

```
{
  "_id": { "$oid": "698db76c4a1f5c8fa4936c8d" },
  "name": "Edgar Allan Poe",
  "status": "ACTIVE",
  "published_books": [
    {
      "_id": { "$oid": "698db76c4a1f5c8fa4936c8e" },
      "title": "Tales of Mystery and Imagination"
    },
    {
      "_id": { "$oid": "698db76c4a1f5c8fa49373bf" },
```

```
      "title": "Fall of the House of Usher and Other Stories"
    },
    {
      "_id": { "$oid": "698db76d4a1f5c8fa4937aef" },
      "title": "The Raven And Other Poems"
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa49395df" },
      "title": "The Pit and the Pendulum"
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa4939a9c" },
      "title": "The Narrative of Arthur Gordon Pym of Nantucket"
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa493a884" },
      "title": "Edgar Allan Poe Reader"
    }
  ],
  "ratings_count": 14,
  "sum_ratings": 1234
}
```


Listing 3.4: Example Author Document (Edgar Allan Poe)

Authors Collection

The authors collection is designed to facilitate quick retrieval of author profiles and their complete bibliography. It utilizes the **Extended Reference Pattern** to list published books and the **Computed Pattern** to maintain real-time aggregate statistics.


```
{
  "_id": { "$oid": "698db76c4a1f5c8fa4936c8d" },
  "name": "Edgar Allan Poe",
  "status": "ACTIVE",
  "published_books": [
    {
      "_id": { "$oid": "698db76c4a1f5c8fa4936c8e" },
      "title": "Tales of Mystery and Imagination"
    },
    {
      "_id": { "$oid": "698db76c4a1f5c8fa49373bf" },
      "title": "Fall of the House of Usher and Other Stories"
    },
    {
      "_id": { "$oid": "698db76d4a1f5c8fa4937aef" },
      "title": "The Raven And Other Poems"
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa49395df" },
      "title": "The Pit and the Pendulum"
```

```
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa4939a9c" },
      "title": "The Narrative of Arthur Gordon Pym of Nantucket"
    },
    {
      "_id": { "$oid": "698db76e4a1f5c8fa493a884" },
      "title": "Edgar Allan Poe Reader"
    }
  ],
  "ratings_count": 14,
  "sum_ratings": 1234
}
```

Listing 3.5: Example Author Document (Edgar Allan Poe)

3.3 Graph Store: Neo4j

Neo4j is utilized strictly for modeling relationships and enabling high-performance traversal queries. The graph schema is designed to be lightweight: nodes primarily contain IDs linking back to MongoDB, while relationships carry timestamp data to support temporal queries.

3.3.1 Nodes and Labels

- **User:** Contains `mongoId`, `username`, `country`.
- **Book:** Contains `mongoId`, `title`, `year`.
- **Review:** Contains `mongoId`, `rating`, `createdAt`.
- **Author/Genre:** Lightweight nodes for structural categorization.

3.3.2 Relationships

Specific relationships are modeled to support the recommendation engine:

- `(:User)-[:FOLLOWS {since: ...}]->(:User)`: the social graph backbone.
- `(:User)-[:LIKES {timestamp: ...}]->(:Book)`: a user like a book.
- `(:User)-[:POSTED {timestamp: ...}]->(:Review)`: a user posted a review.
- `(:User)-[:LIKES {timestamp: ...}]->(:Review)`: a user like a review of another user.
- `(:User)-[:LIKES {timestamp: ...}]->(:Author)`: a user like an author.

- `(:User)-[:LIKES {timestamp: ...}]->(:Genre)`: a user like a book's genre.
- `(:Review)-[:REFER_TO]->(:Book)`:connect a review with the book reviewed.
- `(:Book)-[:BELONGS_TO]->(:Genre)`: a book belongs to one ore more genres.
- `(:Author)-[:WROTE]->(:Book)`: a author is the writer of a certain book.

3.4 Polyglot Persistence Strategy

Maintaining data integrity between a Document Store and a Graph Database requires a strictly defined synchronization strategy. BookSphere prioritizes Eventual Consistency for high-frequency social features (Likes, Views) to ensure system availability, while enforcing Strict Consistency for critical Admin and Moderation operations.

3.4.1 Synchronization Matrix

The following table details how operations are propagated across the two databases:

3.4.2 Error Handling Strategy

To ensure robustness, we implemented a specific error handling flow:

- **MongoDB Failure:** If the write to the primary DB fails, the operation aborts immediately. No try-catch block is used, allowing the `@Transactional` annotation to handle the rollback automatically.
- **Neo4j Failure:** If the write to MongoDB succeeds but Neo4j fails, a specific `try-catch` block intercepts the error and performs a manual rollback on MongoDB (deleting the just-created entity) to prevent "phantom data" and maintain consistency.

Chapter 4

Implementation

4.1 Technologies and Frameworks

The application is built using Java and the **Spring Boot** framework, chosen for its robust ecosystem and native support for building RESTful web services. To manage the interaction with the NoSQL databases, the project relies on **Spring Data MongoDB** and the **spring-boot-starter-data-neo4j** module, which seamlessly integrates the Neo4j Java driver. Furthermore, the application employs **MapStruct** for automated object mapping and **JSON Web Tokens (JWT)** to enforce stateless, role-based security.

Additionally, tools such as **Lombok** have been utilized to significantly reduce boilerplate code and facilitate secure, constructor-based dependency injection. Regarding security, the application leverages Spring Security's dedicated and integrated features to reinforce overall system protection. Finally, the project adopts a strict layered architecture, which is thoroughly described in Section 4.2.

4.2 Directory Structure and Modules

To ensure maintainability, scalability, and a clear separation of concerns, the codebase is organized following a strict Layered Architecture. The main packages and their responsibilities are detailed below.

4.2.1 Config

The `Config` module groups all the configuration classes required to bootstrap and secure the application environment. Key components include:

- `SecurityConfig`: Defines the role-based access rules and permissions for every exposed REST endpoint.
- `JwtAuthenticationFilter`: A custom filter that intercepts incoming HTTP requests to validate the signature and expiration (set to 24 hours) of the provided JWT.

- `JwtAuthenticationEntryPoint`: Manages authentication failures, gracefully handling unauthorized access attempts by returning standardized HTTP 401 (Unauthorized) responses.
- `DatabaseConfig`: Creates the transactional managers for the two databases.

4.2.2 Controllers

This package contains the REST controllers responsible for exposing the application's endpoints and handling incoming HTTP requests. To ensure a clear separation of concerns, these controllers delegate all the underlying business logic to the Service layer, thereby maintaining the presentation layer as lightweight and efficient as possible.

The API endpoints are logically divided into three main categories based on their access level: open (public APIs), registered (user-specific APIs, distinguished by the `/me/` prefix), and administrative (distinguished by the `/admin/` prefix).

Public Controllers

These controllers handle requests that do not require user authentication:

- **AuthController**: Manages user authentication, including login and logout functionalities.
- **AnalyticsController**: Implements the main system analytics and statistics, accessible without registration[cite: 19].
- **BookController**: Allows users to search for and retrieve book details by ID or title.
- **AuthorController**: Handles queries related to authors (e.g., retrieving author details or searching by name).
- **UserController**: Provides functionalities to search for registered users by their ID or username.
- **ReviewsController**: Fetches multiple reviews based on a provided list of identifiers.

Registered User Controllers (`/me/`)

These controllers expose functionalities reserved for authenticated users:

- **BookshelfController**: Enables users to manage their personal bookshelves (add, modify, or remove books).
- **ReviewController**: Allows authenticated users to create, update, or delete their personal reviews.

- **FollowController**: Manages the social graph, allowing users to follow/unfollow other users or retrieve the list of followed accounts.
- **LikeController**: Handles the interactions for liking or unliking specific content (e.g., reviews or books).
- **ProfileController**: Manages account settings, providing options to update the username or delete the account entirely.
- **UserFeaturesController**: Implements user-specific analytics and custom features, such as personalized book recommendations or yearly summaries (e.g., "Wrapped").

Admin Controllers (`/admin/`)

These controllers are restricted to users with administrative privileges:

- **AdminBookController** & **AdminCatalogController**: Used by administrators to manage the application's catalog, including adding or updating books, authors, and genres.
- **AdminModerationController**: Provides moderation tools to ban malicious users or remove inappropriate reviews.

Endpoint Architecture and Validation

As recommended by the project guidelines, we adopted the Spring framework to implement the RESTful APIs. Furthermore, annotations such as `@Operation` are used to seamlessly integrate with Swagger for complete API documentation[cite: 52, 53].

The following snippet illustrates a typical controller method, demonstrating how requests are received, validated, and passed to the Service layer:

```
@Operation(
    summary = "Update book status",
    description = "Change the status of a book in the bookshelf"
)
@PatchMapping("/{bookID}")
public ResponseEntity<Map<String, String>> updateBookStatus(
        @Parameter(description = "MongoDB ObjectId of the book",
           example = "65b3f...")
        @PathVariable String bookID,
        @Parameter(description = "New status", example = "read")
        @RequestBody Map<String, String> requestBody
) {
    // Input validation performed at the controller level
    ValidationUtils.validateObjectId(bookID, "bookID");
    String status = ValidationUtils.extractAndValidateField(
       requestBody, "status");
    ValidationUtils.validateBookshelfStatus(status);

    // Delegation to the Service layer
```

```
        bookshelfService.updateBookStatus(bookID, status);

        return ResponseEntity.ok(Map.of("message", "Book␣status␣
            updated"));
    }
```

Listing 4.1: Example of a Controller endpoint with input validation (BookshelfController)

As demonstrated in the code above, user input validation is strictly enforced at the controller level prior to any business logic execution. To avoid redundancy in this documentation, we omit further code examples for other endpoints. The architectural pattern remains highly consistent across the application: variations strictly depend on the required CRUD operation [cite: 18] (mapped via standard annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, or `@DeleteMapping`) and the method of parameter extraction (using `@PathVariable`, `@RequestParam`, or `@RequestBody`).

4.2.3 DTO

To decouple the internal domain models (Entities) from the data exposed through the APIs, the application makes extensive use of *Data Transfer Objects* (DTOs), such as the `AuthResponseDTO`.

4.2.4 Mappers

The `Mappers` directory contains the MapStruct interfaces, which automatically generate the boilerplate code required for bidirectional conversion between Entities and DTOs.

4.2.5 Model

Light model description

4.2.6 Repositories

This layer manages all data access operations. It contains interfaces extending `MongoRepository` and `Neo4jRepository`. Besides the standard CRUD operations, the Neo4j repositories include custom `@Query` methods written in Cypher for complex graph traversals (e.g., fetching user recommendations or analytics). The package also includes *Projections*, which are specific interfaces used by Spring Data to capture and map intermediate results returned by complex aggregation queries.

4.2.7 Services and Async

The `Services` package encapsulates the core business logic of the application. It orchestrates the flow of data, applies business rules, and interacts with the repositories. Within this package, a dedicated `Async` sub-directory was created to group all the asynchronous

services. These classes are annotated with `@Async` and are crucial for implementing the *Eventual Consistency* patterns (e.g., updating denormalized review snapshots in the background). DETEAILED DESCRITION(?) AND CODE OF SOME NICE FUNCTION

4.2.8 Utils

The `Utils` package provides shared helper classes and utilities utilized across different layers of the application:

- `JWTUtils`: Responsible for generating tokens and securely extracting claims during the validation phase.
- `UserPrincipal`: A structural class representing the currently authenticated entity (either a standard user or an admin).
- `SecurityUtils`: A utility class designed to easily retrieve the `UserPrincipal` and the current user's details directly from the Spring Security context, avoiding the need to pass the JWT manually through method signatures.

4.2.9 Validation

This module centralizes request payload validation

4.2.10 Exceptions

To guarantee data integrity and provide meaningful feedback to the clients, this module centralizes error handling. It leverages a `GlobalExceptionHandler`

4.3 MongoDB Relevant queries

To fulfill the analytical requirements of the application and handle the large dataset efficiently, several complex aggregation pipelines have been designed for the Document Database. These pipelines heavily leverage MongoDB's native operators to process and transform data directly within the database engine, minimizing data transfer and optimizing performance.

4.3.1 Ranking

This pipeline computes the ranking of books based on their average rating. It filters the statistics by a specific year, aggregates the total ratings, and enforces a minimum threshold to exclude statistical outliers. Finally, it calculates the exact average and sorts the results.

```
1 db.books.aggregate([
2     // 1. Initial Match
```

```
    {
        "$match": {
            "availability": "ACTIVE"
            // "author.id": {$id}, // (Optional)
            // "genres": "Horror"  // (Optional)
        }
    },
    // 2. Filter Array (Year Logic)
    {
        "$project": {
            "title": 1,
            "author": 1,
            "targetStat": {
                "$filter": {
                    "input": "$stats_per_year",
                    "as": "stat",
                    "cond": { "$eq": ["$$stat.year", 2023] }
                }
            }
        }
    },
    // 3. Extract Totals (Flattening)
    {
        "$project": {
            "title": 1,
            "author": 1,
            "totalRatings": { "$sum": "$targetStat.ratings_count"
                },
            "sumRating": { "$sum": "$targetStat.sum_rating" }
        }
    },
    // 4. Threshold Filter
    {
        "$match": { "totalRatings": { "$gt": 5 } }
    },
    // 5. Calculate Average & Rename fields for DTO
    {
        "$project": {
            "name": "$title",
            "additionalInfo": "$author.name",
            "totalRatings": 1,
            "averageRating": { "$divide": ["$sumRating", "
               $totalRatings"] }
        }
    },
    // 6. Final Sort & Limit
    { "$sort": { "averageRating": -1 } },
    { "$limit": 25 }
])
```

Listing 4.2: Book Rankings Aggregation Pipeline

4.3.2 Revaluation

This query identifies the books with the most significant rating gap between their first and last year of reviews. It extracts the first and last statistical objects from the `stats_per_year` array, safely calculates their respective averages avoiding division by zero, and computes the delta.

```
db.books.aggregate([
    // 1. Filter books with at least 2 years of history
    { "$match": {
        "stats_per_year.1": { "$exists": true },
        "availability": "ACTIVE"
    }},
    // 2. Extract first and last stat objects
    { "$project": {
        "title": 1,
        "author": 1,
        "firstStat": { "$arrayElemAt": ["$stats_per_year", 0] },
        "lastStat": { "$arrayElemAt": ["$stats_per_year", -1] }
    }},
    // 3. Calculate Averages (Safe Division: Sum / Count)
    { "$project": {
        "title": 1,
        "author": "$author.name",
        "startYear": "$firstStat.year",
        "endYear": "$lastStat.year",
        "startRating": {
            "$cond": [
                { "$gt": ["$firstStat.ratings_count", 0] },
                { "$divide": ["$firstStat.sum_rating", "
                   $firstStat.ratings_count"] },
                0.0
            ]
        },
        "endRating": {
            "$cond": [
                { "$gt": ["$lastStat.ratings_count", 0] },
                { "$divide": ["$lastStat.sum_rating", "$lastStat.
                   ratings_count"] },
                0.0
            ]
        }
    }},
    // 4. Calculate the Delta (End - Start)
    { "$addFields": {
        "ratingDelta": { "$subtract": ["$endRating", "
           $startRating"] }
    }},
    // 5. Sort and Limit
    { "$sort": { "ratingDelta": -1 } },
    { "$limit": 10 }
```

```
]);
```

Listing 4.3: Book Revaluation Aggregation Pipeline

4.3.3 Yearly Wrapped

This advanced pipeline generates an annual summary for a specific user. It leverages the `$facet` operator to perform multiple parallel analyses—such as extracting the most read authors, the favorite genres, and the best/worst rated books—in a single database roundtrip. The `$unwind` operator is essential here to accurately flatten and count the frequency of array elements (authors and genres) before grouping them.

```
db.users.aggregate([
    // 1. MATCH: Filter by specific user
    { "$match": { "_id": "CURRENT_USER_ID" } },

    // 2. ADDFIELDS: Pre-calculate data (filter and sort internal arrays)
    { "$addFields": {
        "best_book": {
            "$arrayElemAt": [
                { "$sortArray": {
                    "input": { "$ifNull": ["$reviews_year", []] },
                    "sortBy": { "rating": -1 } // DESC
                }}, 0
            ]
        },
        "worst_book": {
            "$arrayElemAt": [
                { "$sortArray": {
                    "input": { "$ifNull": ["$reviews_year", []] },
                    "sortBy": { "rating": 1 } // ASC
                }}, 0
            ]
        },
        "yearly_books": {
            "$filter": {
                "input": { "$ifNull": ["$bookshelf", []] },
                "as": "b",
                "cond": {
                    "$and": [
                        { "$eq": ["$$b.status", "read"] },
                        { "$gte": ["$$b.added_at", ISODate("2026-01-01T00:00:00Z")] },
                        { "$lte": ["$$b.added_at", ISODate("2026-12-31T23:59:59Z")] }
                    ]
                }
```

```
            }
        }
    }},

    // 3. FACET: Execute 3 parallel sub-pipelines
    { "$facet": {
        // Analysis 1: Top Authors
        "top_authors": [
            { "$project": { "yearly_books": 1 } },
            { "$unwind": "$yearly_books" },
            { "$group": {
                "_id": "$yearly_books.author.name",
                "count": { "$sum": 1 }
            }},
            { "$sort": { "count": -1 } },
            { "$limit": 3 }
        ],
        // Analysis 2: Top Genres
        "top_genres": [
            { "$project": { "yearly_books": 1 } },
            { "$unwind": "$yearly_books" },
            { "$unwind": "$yearly_books.genres" },
            { "$group": {
                "_id": "$yearly_books.genres",
                "count": { "$sum": 1 }
            }},
            { "$sort": { "count": -1 } },
            { "$limit": 3 }
        ],
        // Analysis 3: Metadata (Best/Worst & Totals)
        "meta": [
            { "$project": {
                "best_book": 1,
                "worst_book": 1,
                "total_books_read": { "$size": "$yearly_books" }
            }}
        ]
    }}
])
```

Listing 4.4: Yearly Wrapped Aggregation Pipeline

4.4 Neo4j Relevant Queries

To leverage the highly connected nature of the dataset and satisfy the requirement for typical ”on-graph” domain-specific queries, several advanced Cypher queries have been implemented. These queries navigate complex, multi-hop paths to calculate metrics and provide social features that would be computationally prohibitive in a standard Document DB.

4.4.1 User Recommendations

This query provides personalized book suggestions by traversing a 2-to-3-hop path. The algorithm recommends books based on three main social factors: books liked by the users the current user follows, books written by authors the user likes, and books belonging to genres the user likes. To optimize the recommendation engine and prevent suggesting books the user has already read, a pre-query is executed on the MongoDB backend to extract the read books from the user's `bookshelf`. This list is then passed to Neo4j as a blacklist parameter (`$excludedMongoIds`).

```
MATCH (u:User {mongoId: $userId})
OPTIONAL MATCH (u)-[:FOLLOWS]->(:User)-[:LIKES]->(b1:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Author)-[:WROTE]->(b2:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Genre)<-[:BELONGS_TO]-(b3:Book)
WITH collect(b1) + collect(b2) + collect(b3) AS recommendations, u
UNWIND recommendations AS book
WITH u, book
WHERE NOT EXISTS((u)-[:POSTED]->(:Review)-[:REFER_TO]->(book)) AND book
    IS NOT NULL
WITH book, count(*) AS score
ORDER BY score DESC
LIMIT $limit
MATCH (a:Author)-[:WROTE]->(book)
RETURN book.mongoId AS bookId,
       book.title AS title,
       book.year AS publicationYear,
       score,
       collect(a.name) AS authors
```

Listing 4.5: Cypher Query for User Recommendations

4.4.2 Internationality Index

This analytic calculates the "global reach" (Internationality Index) of a Book or an Author by analyzing the geographic origin (`country` property) of the users who interact with the entity. The traversal captures both users who posted a review and users who simply left a "Like".

```
// Internationality Index for a Book
MATCH (b:Book {mongoId: $bookId})
OPTIONAL MATCH (b)<-[:REFER_TO]-(r:Review)<-[:POSTED]-(reviewer:User)
OPTIONAL MATCH (b)<-[:LIKES]-(liker:User)
WITH reviewer, liker
WITH collect(DISTINCT reviewer) + collect(DISTINCT liker) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
       count(DISTINCT u.mongoId) AS uniqueUsers,
       count(*) AS totalInteractions
ORDER BY uniqueUsers DESC

// Internationality Index for an Author
MATCH (a:Author {mongoId: $authorId})
OPTIONAL MATCH (a)<-[:LIKES]-(liker:User)
```

```
OPTIONAL MATCH (a)-[:WROTE]->(b:Book)<-[:REFER_TO]-(r:Review)<-[:POSTED
    ]-(reviewer:User)
WITH liker, reviewer
WITH collect(DISTINCT liker) + collect(DISTINCT reviewer) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
       count(DISTINCT u.mongoId) AS uniqueUsers,
       count(*) AS totalInteractions
ORDER BY uniqueUsers DESC
```

Listing 4.6: Cypher Query for Book and Author Internationality Index

4.4.3 Genre Influencer

This query aims to identify "True Influencers" within the platform, focusing on quality over quantity. It searches for users who write reviews that receive a high volume of "Likes" from other users, either globally or filtered by a specific genre. To prevent anomalies, the query enforces a minimum threshold of reviews and excludes soft-deleted or banned users (represented by an empty string username).

```
// Influencers by a specific genre
MATCH (g:Genre {name: $genreName})<-[:BELONGS_TO]-(b:Book)<-[:REFER_TO
    ]-(r:Review)<-[:POSTED]-(influencer:User)
MATCH (r)<-[:LIKES]-(fan:User)
WITH influencer,
     count(DISTINCT r) AS numReviews,
     count(fan) AS totalLikes
WHERE numReviews > 3 AND influencer.username <> ""
RETURN influencer.username AS username,
       totalLikes AS totalEngagement,
       numReviews AS numReviews,
       toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit

// Global Influencers (All genres)
MATCH (r:Review)<-[:POSTED]-(influencer:User)
MATCH (r)<-[:LIKES]-(fan:User)
WITH influencer,
     count(DISTINCT r) AS numReviews,
     count(fan) AS totalLikes
WHERE numReviews > 5 AND influencer.username <> ""
RETURN influencer.username AS username,
       totalLikes AS totalEngagement,
       numReviews AS numReviews,
       toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit
```

Listing 4.7: Cypher Query for Genre Influencers

4.5 API Documentation

4.5.1 API endpoint

LIST OF ENDPOINT

4.5.2 API Examples

ONE OR TWO EXAMPLES

Chapter 5

Databases Choices and Distributed Architecture

5.1 Polyglot Persistence Strategy

The application adopts a Polyglot Persistence approach to efficiently manage a highly connected, large-scale dataset. MongoDB was selected as the Primary Document Database to store the core entities (Users, Books, Authors, and Reviews) due to its high throughput in read/write operations and its flexible schema. Conversely, Neo4j was introduced as a Secondary Graph Database to seamlessly compute complex social interactions, recommendations, and graph-centric analytics (e.g., the Author Versatility Index).

5.2 Indexes Justification and Validation

To guarantee optimal response times and avoid full collection scans (`COLLSCAN`), specific indexes have been designed and experimentally validated for the Document DB. On the Graph DB, indexes were defined strictly where needed to optimize traversal starting points.

5.2.1 MongoDB Indexes

STILL TO CHOOSE TO CHOOSE, for sure on genres on book

5.2.2 Neo4j Indexes

In Neo4j, index usage was minimized to avoid unnecessary write overhead. A *Range Index* was created exclusively on the `mongoId` property across all primary nodes (`User`, `Book`, `Review`, `Author`) and also for `name` for `Genre` node. This is crucial to guarantee $O(\log n)$ lookup times during find query, synchronization and cross-database queries.

5.3 Virtual Machine Organization and Replica Sets

To guarantee high availability and fault tolerance, the database layer was deployed both on a local cluster and on the VIRTUAL LAB cluster provided by UNIPI.

5.3.1 Document Database Deployment

For the Document DB, a distributed architecture comprising a three-member replica set has been defined and configured to manage eventual consistency. This standard topology consists of one *Primary* node, which handles all write operations, and two *Secondary* nodes that asynchronously replicate the primary's oplog to ensure data redundancy.

The replica set initialization configuration (`rsconf`) assigns specific priorities to the nodes to govern the primary election process. As shown in the configuration below, the server hosted at `10.1.1.80` is explicitly designated as the preferred primary node due to its higher priority value:

```
var rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.77", priority: 1 },
    { _id: 1, host: "10.1.1.78", priority: 2 },
    { _id: 2, host: "10.1.1.80", priority: 5 }
  ]
};
```

Listing 5.1: MongoDB Replica Set Configuration (`rsconf`)

5.3.2 Graph Database Deployment

Regarding the Graph DB, deploying replicas on the virtual cluster is not mandatory. Since configuring a native Neo4j cluster requires an enterprise license, a single instance of the graph database was deployed on the virtual cluster alongside the Document DB replicas. To optimize resource allocation and prevent interference with the primary MongoDB node's heavy write workloads, the Neo4j instance was strategically co-located on one of the secondary servers.

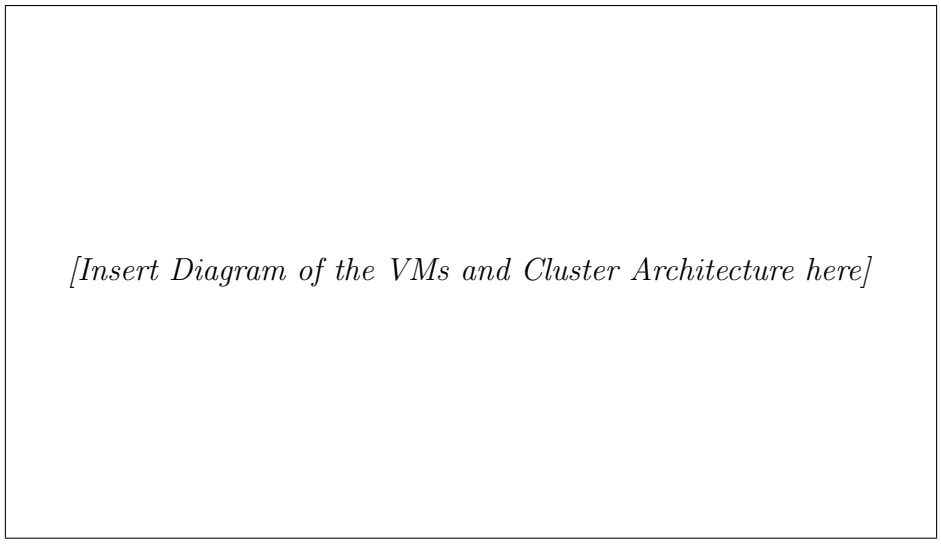


Figure 5.1: Overview of the Virtual Machines Organization and Database Deployment on the UNIPI Cluster.

5.4 Sharding Strategy

As the dataset grows, horizontal scaling becomes necessary. The design of the sharding strategy for MongoDB was argued and planned to distribute the load evenly across the cluster. The `Review` collection is the fastest-growing entity in the system. To prevent "jumbo chunks" and ensure a uniform distribution of read/write operations, a **Hashed Sharding** strategy was chosen. By sharding on the hashed value of the `_id` or the `user_id`, incoming reviews are evenly distributed across the shards, effectively preventing localized hotspots during traffic spikes. TO ADD OTHER THINGS, for exemple partition on other enteties like BOOK, AUTHOR and User

5.5 CAP Theorem Considerations

Considerations regarding the features of the application in relation to the CAP theorem (Consistency, Availability, Partition Tolerance) were carried out after deploying the database on the cluster. By default, a MongoDB replica set strongly favors **CP** (Consistency and Partition Tolerance). If the primary node fails, the system becomes temporarily unavailable for writes until a new primary is elected. However, to enhance *Availability* for read-heavy operations (e.g., fetching book catalogs or public reviews), the application leverages *Read Preferences* (setting them to `secondaryPreferred`). This shifts the system towards an **AP** model for read operations, accepting *Eventual Consistency* in exchange for higher availability and lower latency. Mongo DB automatically managed the eventual consistency (w=majority), when set the replicas will be acknowledged of the writes before committed.

5.6 Inter-Database Consistency

Maintaining consistency between the different database architectures is a critical challenge in polyglot systems. Since distributed Atomic transactions are not supported between MongoDB and Neo4j, the application enforces consistency through two main strategies:

- **Compensating Transactions (Rollbacks):** For critical operations, such as User Registration, a synchronous approach is used. If the Neo4j node creation fails, a manual rollback removes the previously inserted MongoDB document.
- **Eventual Consistency:** For high-volume, non-critical operations (like updating "Likes" or recalculating average ratings), the system relies on Spring's `@Async` workers. Data is immediately persisted in the primary database, while the secondary database or denormalized snapshots are updated asynchronously in the background.

5.6.1 Eventual Consistency via @Async

To prevent bottlenecks during high-frequency write operations, the updating of denormalized data strictly follows the **Eventual Consistency** paradigm. Through the `@Async` annotation, non-critical operations are decoupled from the main HTTP request thread. For instance, the propagation of the "popular reviews" embedding within book documents, as well as the recalculation of aggregate statistics, are processed asynchronously.

5.6.2 Resilience with @Retryable

To ensure system reliability in a distributed environment prone to temporary network latencies, **Spring Retry** (`@Retryable`) has been enabled. This allows the system to automatically retry failed operations. Special care was taken to ensure the *idempotency* of these queries, guaranteeing that multiple re-executions do not generate inconsistent states.

Chapter 6

System Testing and Performance Evaluation

6.1 Introduction

In this chapter, we aim to exhaustively describe the system's testing procedures. This validation strategy covers two primary pillars:

1. **Java-based Testing:** Comprehensive unit and integration tests covering all internal code functionalities.
2. **API Performance Testing:** External load and stress testing performed via Postman (and Newman CLI) to validate system resilience under concurrency.

The following sections detail the methodology, configuration parameters, and quantitative results of these tests.

6.2 Java-based Testing Strategy

This section outlines the testing framework applied to the Java backend. We utilize standard testing libraries to ensure code reliability and logic correctness before deployment.

6.2.1 Unit Testing

6.2.2 Integration Testing

6.3 API Performance Testing via Postman

To evaluate the system's behavior under stress, we conducted a series of performance tests targeting specific API endpoints. These tests are divided into **Read Performance** (data retrieval) and **Write Performance** (data ingestion/modification).

For each test scenario, we define a specific **Test Setup** configuration encompassing:

- **Virtual Users (VU):** The number of concurrent simulated users accessing the system.
- **Duration:** The total timeframe over which the test is executed.
- **Load Profile:** The intensity and distribution of traffic (e.g., fixed, ramp-up).

6.3.1 Read Performance Scenarios

This section details the performance metrics for data retrieval operations. The primary goal is to measure latency and throughput when multiple users query the application simultaneously.

Test Setup: Read Operations

Table 6.1: Configuration for Read Operation Load Test

Parameter	Configuration Value
Virtual Users	50 VUs (Simulated)
Duration	10 Minutes
Load Profile	Ramp-up (0 to 50 over 1 min)
Endpoint Tested	`GET /api/v1/resource`

Results and Analysis

The results of the read performance test indicate the system's capability to handle high-concurrency read requests. As shown in Figure 6.1, the response time remained stable even as the number of virtual users peaked.

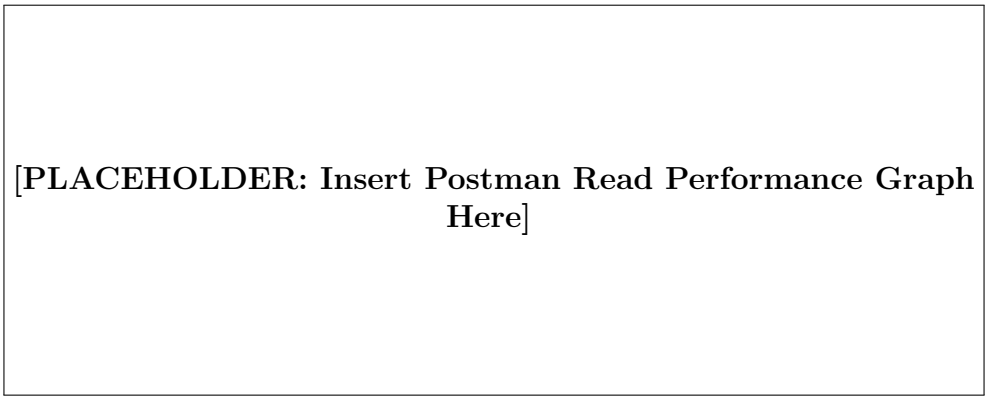


Figure 6.1: Postman Response Time Graph for Read Operations

6.3.2 Write Performance Scenarios

This section evaluates the system's performance during data ingestion. Write operations generally incur higher overhead due to database transaction management and constraint checking.

Test Setup: Write Operations

Table 6.2: Configuration for Write Operation Load Test

Parameter	Configuration Value
Virtual Users	20 VUs (Simulated)
Duration	5 Minutes
Load Profile	Fixed Load
Endpoint Tested	`POST /api/v1/resource`

Results and Analysis

The write performance metrics demonstrate the system's throughput for transactional operations. Figure 6.2 highlights the relationship between the active virtual users and the server's processing time per write request.

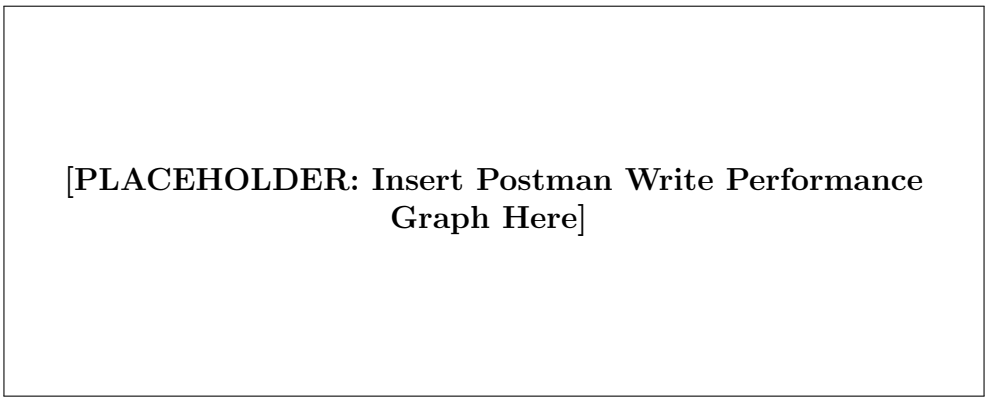


Figure 6.2: Postman Throughput Graph for Write Operations

6.4 Conclusion

The combination of granular Java unit tests and broad Postman load tests provides high confidence in the system's stability.